

INFORMATION SECURITY, GAME SECURITY & IT SUPPORT PORTFOLIO

CONTACT
Email: timberlyaustin@gmail.com
Phone Number: 870-317-1152

Timberly Austin
Aspiring Cybersecurity Professional

Areas of Interest: Game Security, Cybersecurity, IT Support, Security Analytics, Secure Development

About me

I am aspiring cybersecurity professional developing hands-on skills in security analysis, Python programming, cybersecurity frameworks, and secure software development, and IT support. I am especially interested in information security within the gaming industry, including game security, player security, threat detection, risk management, and technical support. This portfolio documents hands-on projects I developed to strengthen my technical, troubleshooting, and cybersecurity skills while exploring technology challenges relevant to gaming environments.

Programming

- Python

Data & Analysis

- Pandas
- Matplotlib
- CSV data analysis
- Player telemetry analysis
- Data visualization
- Rule-based risk classification

Cybersecurity

- Risk assessment
- Security monitoring
- Threat detection
- Windows Security Event Log Analysis
- Authentication and access control
- Threat modeling
- Security control selection
- NIST Cybersecurity Framework 2.0

Windows Security

- Windows Security Event Log analysis
- Windows Event ID analysis

- PowerShell
- Windows Command Prompt
- Security event detection
- Automated reporting

Web & Application Development

- Flask
- HTML & CSS
- SQLite
- Authentication & session management
- Password hashing
- Secure application development
- Protected routes
- Parameterized SQL queries

Game Security

- Anti-cheat concepts
- Player telemetry analysis
- Behavioral detection
- Game-security threat modeling
- Game-integrity risk analysis
- Multiplayer security architecture

IT & Desktop Support

- Windows system diagnostics
- Hardware and driver review
- Network connectivity troubleshooting
- DNS troubleshooting
- System performance monitoring
- Storage and system health review
- Command-line diagnostics
- Technical documentation
- Structured troubleshooting
- Customer-focused technical support

PROJECT 01

Anti-Cheat Analytics System

Python| Pandas| Matplotlib| Security Analytics

Project Overview

I developed an educational Python-based anti-cheat analytics prototype that analyzes synthetic player telemetry and identifies potentially suspicious gameplay behavior using predefined detection thresholds. The system analyzes player accuracy, kill/death ratio, movement speed, and actions per second. Players are assigned to Low, Medium, or High-risk levels, and the system reports the indicators responsible for each classification.

Objective

Build a rule-based security analytics tool that can process player telemetry, identify predefined suspicious behavioral indicators, explain its findings, and produce an analysis report.

Technologies Used

- Python: detection logic and automation
- Pandas: reading and analyzing player data
- Matplotlib: creating data visualizations
- CSV: storing simulated player telemetry
- Windows Command Prompt: executing and testing the program

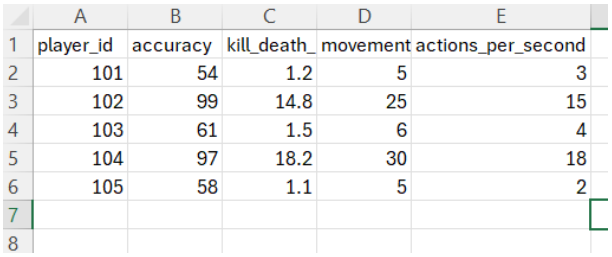
Player Telemetry

Metric	Purpose
Player ID	Identifies each simulated player
Accuracy	Represents shot accuracy
K/D Ratio	Represents kill relative to deaths
Movement Speed	Represents player's movement behavior
Action Per Second	Represents frequency of player actions

Detection Logic

Player Metric	Prototype threshold
Accuracy	>95%
Kill/Death Ratio	>10
Movement Speed	>20
Actions Per Second	>12

Note: The player data and detection threshold used in this project are synthetic and were created for educational purposes. They do not represent a commercial anti-cheat system.



	A	B	C	D	E
1	player_id	accuracy	kill_death_	movement	actions_per_second
2	101	54	1.2	5	3
3	102	99	14.8	25	15
4	103	61	1.5	6	4
5	104	97	18.2	30	18
6	105	58	1.1	5	2
7					
8					

Figure 1: Synthetic player telemetry dataset analyzed by the program.

```
risk_score = 0
reasons = []

if player["accuracy"] > 95:
    risk_score += 1
    reasons.append("Accuracy exceeds threshold")

if player["kill_death_ratio"] > 10:
    risk_score += 1
    reasons.append("Kill/death ratio exceeds threshold")

if player["movement_speed"] > 20:
    risk_score += 1
    reasons.append("Movement speed exceeds threshold")

if player["actions_per_second"] > 12:
    risk_score += 1
    reasons.append("Actions per second exceed threshold")
```

Figure 2: Python detection logic used to evaluate behavioral indicators.

Detection Results

```
C:          ti\OneDrive\Game Security Portfolio\Anti-Cheat Analytics System>python anti_cheat.py
--------------------
Player: 101.0
Risk Level: LOW
Reasons: No suspicious indicators detected
--------------------
Player: 102.0
Risk Level: HIGH
Reasons:
- Accuracy exceeds threshold
- Kill/death ratio exceeds threshold
- Movement speed exceeds threshold
- Actions per second exceed threshold
--------------------
Player: 103.0
Risk Level: LOW
Reasons: No suspicious indicators detected
--------------------
Player: 104.0
Risk Level: HIGH
Reasons:
- Accuracy exceeds threshold
- Kill/death ratio exceeds threshold
- Movement speed exceeds threshold
- Actions per second exceed threshold
--------------------
Player: 105.0
Risk Level: LOW
Reasons: No suspicious indicators detected
```

Figure 3: Successful execution showing player risk classification and the indicators responsible for each result.

Player Accuracy Visualization

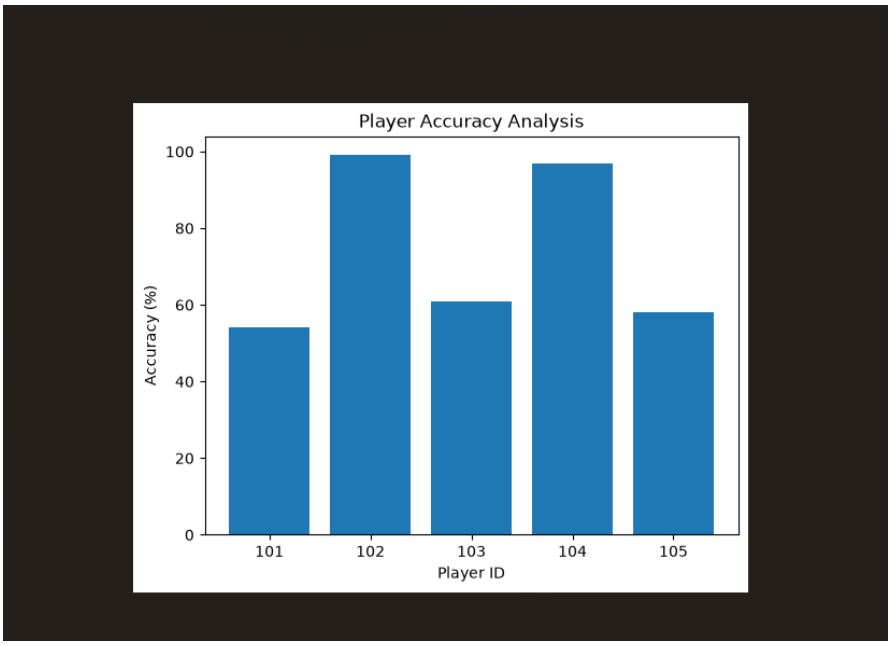


Figure 4: Player accuracy visualization generated using Matplotlib.

Note: Visualizing the telemetry made unusually high values easier to identify. The program combines these observations with additional behavioral indicators rather than relying on accuracy alone.

What I learned

- Reading and analyzing CSV data with Pandas
- Using Python variables, loops, lists, and conditional logic
- Developing rule-based security detection logic
- Creating automated analysis reports
- Visualizing telemetry using Matplotlib
- Executing Python programs through Windows Command Prompt
- Debugging Python and data-related errors

Troubleshooting Experience

Debugging: During development, I encountered IndentationError and a Pandas KeyError caused by a mismatch between a CSV column name and the Python Code. I used the error messages to identify and correct both problems before successfully executing the program.

Security Consideration

A high-risk classification does not prove that a player is cheating. Behavioral anomalies may have legitimate explanations, so detections should support further investigation rather than automatically determine guilt. A production anti-cheat system would require more telemetry, testing, validation, and sophisticated detection methods.

PROJECT 02

Secure Gaming Account Application

Python |Flask |SQLite |HTML |CSS |Secure Authentication

Project Overview

I developed a full-stack gaming account application designed to demonstrate secure user registration, authentication, database management, and access control. Players can create an account, log in with verified credentials, access a protected dashboard, and securely log out.

The project gave me hands-on experience connecting a web interface to a Python backend while incorporating cybersecurity principles into the application.

Tools & Technologies

- Python for backend application logic
- Flask for web application routing and form processing
- SQLite for user account storage
- HTML for webpage structure
- CSS for interface design and styling
- Werkzeug for password hashing and verification
- Windows Command Prompt for running and testing the application

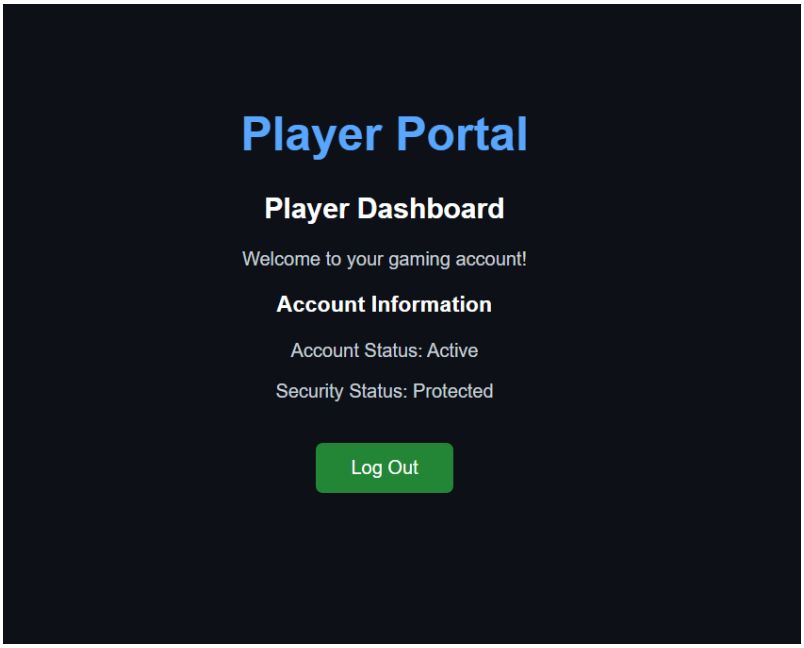


Figure 1: Authenticated Player Dashboard

The protected Player Dashboard is displayed after successful authentication. Access requires an active authenticated session.

Security Test	Result
Valid player registration	PASS
Duplicate username rejected	PASS
Password mismatch rejected	PASS
Correct credentials accepted	PASS
Incorrect credentials rejected	PASS
Unauthorized Dashboard access blocked	PASS
Logout clears authenticated session	PASS
Dashboard blocked after logout	PASS

```
password_hash = generate_password_hash(password)
```

```
connection.execute(
    "INSERT INTO users (username, password_hash) VALUES (?, ?)",
    (username, password_hash)
)
```

Figure 2: Secure Registration Logic

Registration logic hashes player passwords before database storage and uses parameterized SQL queries when inserting account information.

```
if user and check_password_hash(user[2], password):
    session["user_id"] = user[0]
    session["username"] = user[1]
```

Figure 3: Authentication and Session Management

The application verifies the submitted password against the stored hash and creates an authenticated session after successful verification.

What I learned

This project strengthened my understanding of full-stack development and secure authentication. I gained hands-on experience connecting HTML forms to a Python/Flask backend, storing application data in SQLite, hashing and verifying passwords, managing authenticated sessions, protecting application routes, and testing security controls. Troubleshooting was also an important part of the project. I used Python tracebacks and Flask server messages to identify and correct syntax errors, missing imports, HTTP method issues, incorrect HTML, stylesheet-loading problems, database issues, and session configuration errors. This strengthened my debugging and problem-solving skills.

Project Scope: This application is an educational security prototype developed and tested locally. Additional security controls and production configuration would be required before deployment in a real-world environment.

PROJECT 03

Python| Windows Security Logs| PowerShell| Detection Engineering| Automated Reporting

Project Overview

I developed a Python-based Windows security monitoring tool that analyzes real Windows Security Event Log data. The tool queries selected Windows security events, classify the activity using detection rules, displays security findings, and automatically generates a monitoring report.

Objective

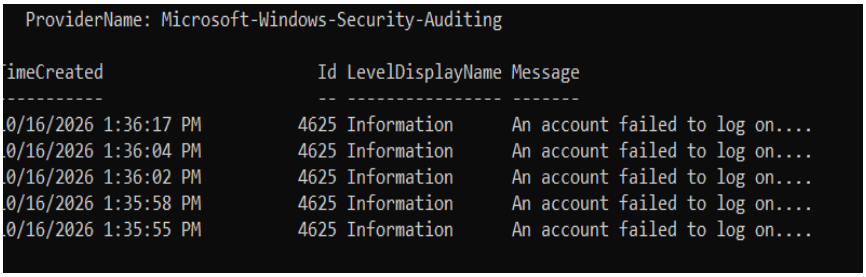
Building a lightweight security monitoring tool that demonstrates how Windows event telemetry can be analyzed with Python to identify authentication failures and monitor process activity.

Technologies Used

- Python for monitoring and detection logic
- Windows Security Event Logs for endpoint telemetry
- PowerShell for event-log validation
- Wetutil for querying Windows events from Python
- Windows Command Prompt for executing and testing the monitor
- Text reporting for automatically documenting findings

Event ID	Meaning	Treatment
4625	Failed logon	Review recommended
4688	New process created	Monitor

Event activity is not automatically proof of malicious behavior. The tool classifies failed authentication as activity requiring review while treating process creation as monitoring telemetry that requires additional context.

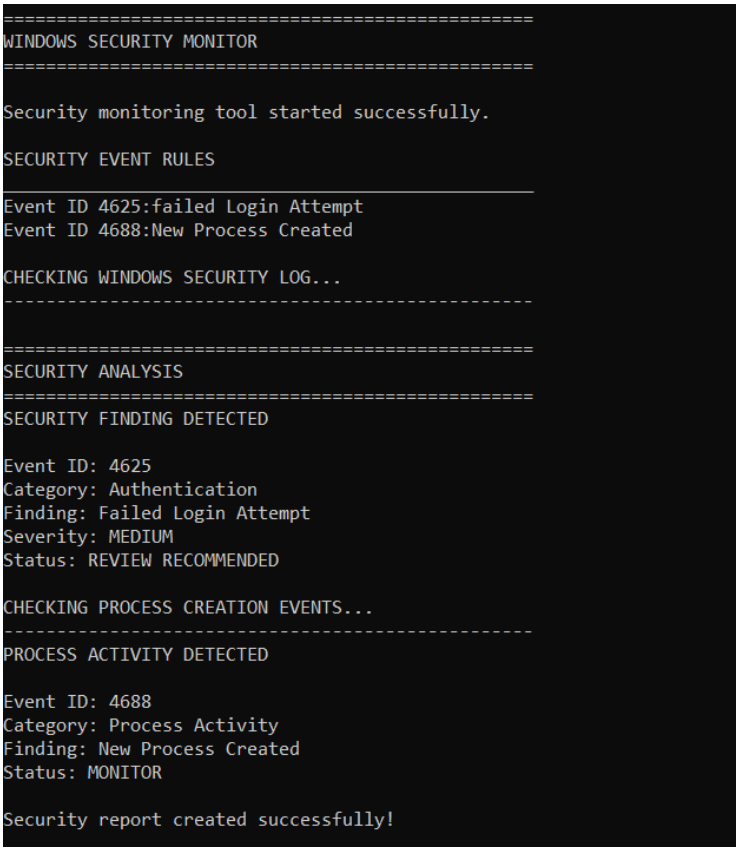


```
  ProviderName: Microsoft-Windows-Security-Auditing

imeCreated                     Id LevelDisplayName Message
----------                     -- ---------------- -------
0/16/2026 1:36:17 PM         4625 Information      An account failed to log on....
0/16/2026 1:36:04 PM         4625 Information      An account failed to log on....
0/16/2026 1:36:02 PM         4625 Information      An account failed to log on....
0/16/2026 1:35:58 PM         4625 Information      An account failed to log on....
0/16/2026 1:35:55 PM         4625 Information      An account failed to log on....
```

Figure 1: Windows Security Event Data

PowerShell was used to validate the presence of Windows Security Event ID 4625 before incorporating the event into the Python monitoring tool.



```
==================================================
WINDOWS SECURITY MONITOR
==================================================

Security monitoring tool started successfully.

SECURITY EVENT RULES
--------------------------------------------------
Event ID 4625:failed Login Attempt
Event ID 4688:New Process Created

CHECKING WINDOWS SECURITY LOG...
--------------------------------------------------

==================================================
SECURITY ANALYSIS
==================================================
SECURITY FINDING DETECTED

Event ID: 4625
Category: Authentication
Finding: Failed Login Attempt
Severity: MEDIUM
Status: REVIEW RECOMMENDED

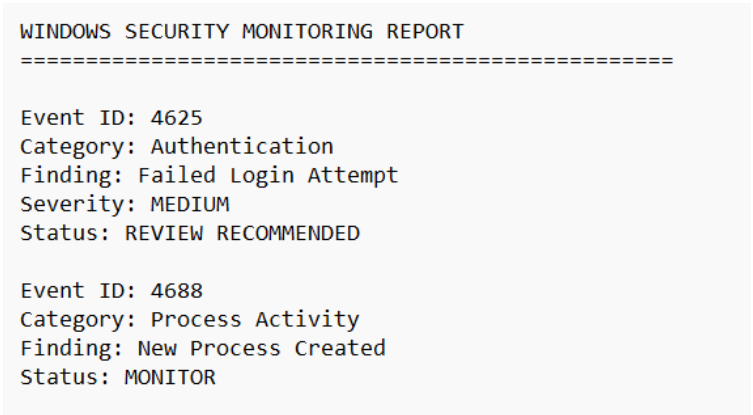
CHECKING PROCESS CREATION EVENTS...
--------------------------------------------------
PROCESS ACTIVITY DETECTED

Event ID: 4688
Category: Process Activity
Finding: New Process Created
Status: MONITOR

Security report created successfully!
```

Figure 2: Automated Security Analysis

The Python monitoring tool evaluates Windows Security events and presents security-relevant activity using predefined detection classifications



```
WINDOWS SECURITY MONITORING REPORT
==================================================

Event ID: 4625
Category: Authentication
Finding: Failed Login Attempt
Severity: MEDIUM
Status: REVIEW RECOMMENDED

Event ID: 4688
Category: Process Activity
Finding: New Process Created
Status: MONITOR
```

Figure 3: Automated Security Monitoring Report

Detected activity is automatically documented in text-based security report for later review.

What I Learned

This project strengthened my understanding of Windows Endpoint security and event-based monitoring. I learned how Windows records security relevant activity, how specific event IDs can represent different types of system activity, and how Python can automate the collection and classification of Windows telemetry. I also strengthened my troubleshooting skills by resolving Python syntax errors, correcting detection-rule event IDS, validating event availability through PowerShell, and testing the completed monitoring workflow.

Project Scope: This is an educational Windows security monitoring protype. Individual Windows events do not independently confirm malicious activity. A production monitoring environment would require broader telemetry, contextual analysis, additional detection rules, tuning, centralized logging, and further validation.

PROJECT4:

Multiplayer Game Threat Model & Cybersecurity Framework Assessment

STRIDE | Risk Assessment | Secure Design | Game Security | Nist CSF 2.0

Project Overview

I designed a fictional multiplayer gaming environment called Project Sentinel and performed a structured cybersecurity assessment of its architecture. I used STRIDE threat modeling to identify potential threats, evaluated those threats through qualitative risk assessment, and developed prioritized security recommendations.

Technologies/ Methodologies

Methods & Frameworks

- STRIDE Threat Modeling
- Qualitative Risk Assessment
- Security Architecture Analysis
- Security Control Selection
- NIST Cybersecurity Framework 2.0

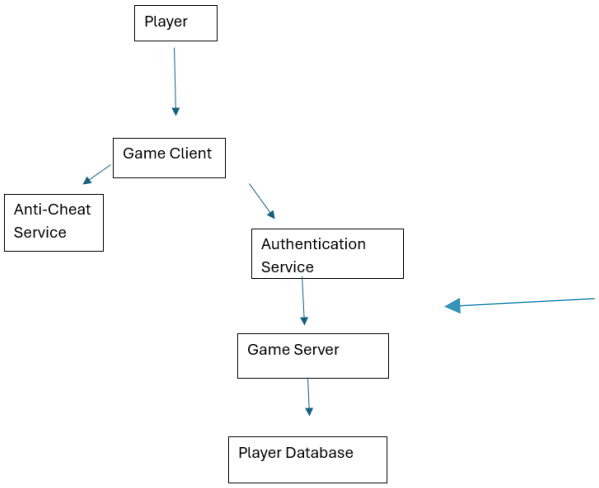


Figure 1: Project Sentinel System Architecture

Simplified fictional multiplayer gaming architecture used to identify system components, communication paths, and potential security threats.

STRIDE	Threat	Component	Concern
Spoofing	Unauthorized account access	Accounts / Authentication	Player impersonation
Tampering	Modification of game data	Client / Server	Game integrity
Repudiation	Denying important actions	Authentication / Server	Lack of accountability
Information Disclosure	Exposure of player information	Database / Authentication	Loss of confidentiality
Denial of Service	Game-service disruption	Server / Authentication	Loss of availability
Elevation of Privilege	Unauthorized permissions	Authentication / Server	Excessive access

Figure 2: STRIDE Threat Analysis

STRIDE was applied to identify potential threats involving authentication, game integrity, accountability, confidentiality, availability, and authorization.

Risk Assessment

The identified STRIDE threats were evaluated using qualitative likelihood and impact ratings. The assessment was used to prioritize risks affecting player accounts, game integrity, player information, service availability, and privileged access.

Threat	STRIDE Category	Likelihood	Impact	Overall Risk
Account Impersonation	Spoofing	High	High	High
Unauthorized game-data modification	Tampering	Medium	High	High
Insufficient activity accountability	Repudiation	Medium	Medium	Medium
Unauthorized player-data exposure	Information Disclosure	Medium	High	High
Game-service disruption	Denial of Service	Medium	High	High
Unauthorized privilege gain	Elevation of Privilege	Medium	High	High

Figure 3: Qualitative Risk Assessment

Identified threats were evaluated using likelihood and impact to establish priorities for recommended security controls.

NIST CSF 2.0 Application

Project Sentinel's security activities were mapped to NIST CSF 2.0 to demonstrate how individual security controls can support a broader cybersecurity risk management strategy.

NIST CSF Function	Project Sentinel Application	Example Activities
Govern	Establish risk management direction	Policies, responsibilities, risk prioritization
Identify	Understand assets and risks	Asset inventory, STRIDE, risk assessment
Protect	Implement safeguards	MFA, access control, encryption, integrity checks
Detect	Identify suspicious activity	Logging, monitoring, anti-cheat telemetry
Respond	Manage security incidents	Investigation, containment, communication
Recover	Restore services	Backups, restoration, lessons learned

Figure 4: NIST CSF 2.0 Mapping

Project Sentinel security activities were organized across Govern, Identify, Protect, Detect, Respond, and Recover.

Recommended Security Priorities

1. **Player Account Security**: MFA, login monitoring, rate limiting.
2. **Game Integrity:** Server-side validation, integrity verification, anti-cheat monitoring
3. **Player Data Protection**: Encryption, access controls, least privilege
4. **Service Resilience**: Availability monitoring, redundancy, recovery planning
5. **Privileged Access**: Role-based access control, access reviews, security logging

What I learned

This project taught me how to move from system architecture to threat identification, risk assessment, control selection, and cybersecurity framework application. I gained practical experience using STRIDE and NIST CSF 2.0 to organize security risks and defensive recommendations for a fictional multiplayer gaming environment.

Project Scope: Project Sentinel is a fictional educational environment. The architecture, risk ratings, and recommendations do not represent the systems or security practice of any real gaming company.

PROJECT 05

IT Desktop Support & Troubleshooting Lab

Windows| Hardware Support| Networking| System Diagnostics| Technical Documentation

Project Overview

I completed a hands-on Windows desktop-support lab focused on structured troubleshooting, system diagnostics, network connectivity, hardware review, performance analysis, storage health, and technical documentation. The project simulated common support scenarios and emphasized a systematic, customer-focused troubleshooting process.

Objective: Develop practical experience using built-in Windows tools and command-line diagnostics to investigate common desktop-support issues and document technical findings.

Technologies & Tools

- Windows System information
- Device Manager
- Task Manger
- Windows Storage Settings
- Command Prompt
- Ipconfig
- Ping
- Nslookup
- Technical troubleshooting documentation

Troubleshooting process

User Reports Issue

Identify Symptoms



Gather System Information

Check Hardware & Devices

↓

Check Network & DNS

↓

Review System Performance

↓

Review storage

↓

Document findings

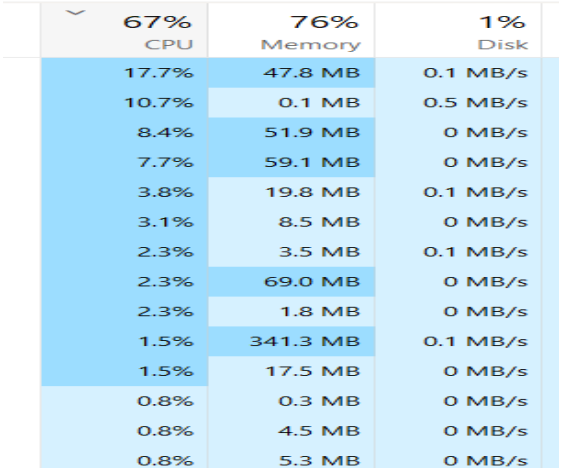
↓

Explain Resolution / Next Steps



Figure 1: Windows Device & Driver Review

Windows Device Manger was used to review installed hardware, network and display adapters, device status, and driver information.



67% CPU	76% Memory	1% Disk
17.7%	47.8 MB	0.1 MB/s
10.7%	0.1 MB	0.5 MB/s
8.4%	51.9 MB	0 MB/s
7.7%	59.1 MB	0 MB/s
3.8%	19.8 MB	0.1 MB/s
3.1%	8.5 MB	0 MB/s
2.3%	3.5 MB	0.1 MB/s
2.3%	69.0 MB	0 MB/s
2.3%	1.8 MB	0 MB/s
1.5%	341.3 MB	0.1 MB/s
1.5%	17.5 MB	0 MB/s
0.8%	0.3 MB	0 MB/s
0.8%	4.5 MB	0 MB/s
0.8%	5.3 MB	0 MB/s

Figure 2: System Performance Review

Windows Task Manager was used to review CPU, memory, disk activity, running processes, and startup applications during performance troubleshooting.

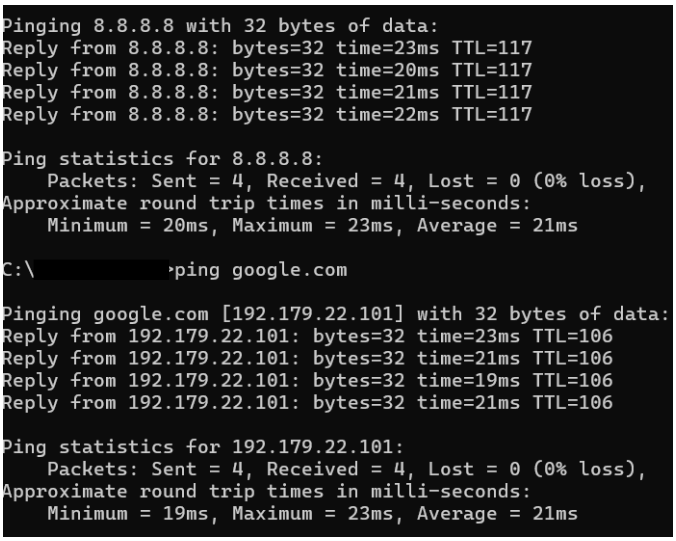


Figure 3: Network Connectivity Testing

Command-line diagnostics were used to verify network connectivity as part of a structured desktop-support troubleshooting workflow.

What I Learned

This project strengthened my understanding of structured Windows desktop troubleshooting. I gained hands-on experience reviewing system information, network configuration, DNS resolution, hardware and driver status, system-resource utilization, startup applications, and storage usage. I also practiced documenting technical findings and approaching support issues systematically rather than making unnecessary system changes before identifying a likely cause.

Project Scope: This educational lab was performed on a personal Windows system using built-in diagnostic tools. Troubleshooting activities focused on observation, diagnosis, and documentation without intentionally creating system failures or making unnecessary configuration changes.